

Trading space for Toffoli depth in Shor’s algorithm with SELECT-SWAP table lookups

Yotam Vaknin*

September 28, 2026

Abstract

Recent high-rate quantum error-correcting codes and large atom arrays are shifting the space–time trade-off for logical qubits: additional logical memory is becoming cheap while the logical clock remains slow, so it can pay to spend substantially more qubits to reduce circuit depth. We apply this trade to the table lookups inside the windowed arithmetic of Shor’s algorithm, replacing the sequential QROM used in current resource estimates by the SELECT-SWAP lookup of Low et al. [8], whose depth shrinks by a tunable factor λ at the price of λ output-sized registers. For RSA-2048 in the Gidney–Ekerå compilation [6], shallow lookups combined with a shallower adder reduce the Toffoli depth by more than an order of magnitude for a few times the logical qubits; the lookup change alone gives a factor of two. We compare this with the other logical depth-for-qubits trade available for RSA-2048, running the residue-arithmetic compilation of Gidney [14] over many primes in parallel as proposed by Webster et al. [5], and find that the latter reaches lower depth at every budget above a few thousand qubits, with fewer Toffolis; the lookup route is the better one only where the exponent register cannot be held. For the 256-bit elliptic-curve discrete logarithm we apply the same change to four published circuits, including a directly measured depth for Schrottenloher’s [12], and find the gain capped at 30–40% of the depth by the serial modular inversion in every one of them, with a simple argument for why lookups are a fixed small fraction of any optimally windowed point addition. The larger lever there is the adder: an estimated variant of Schrottenloher’s circuit with carry-lookahead adders, combined with shallow lookups, would give the lowest ECDLP-256 depth at every qubit budget.

1 Introduction

Resource estimates for Shor’s algorithm have been driven by two metrics, logical qubits and non-Clifford gate count, and the wall-clock time has been taken to be reaction-limited, with one Toffoli resolved per control-system round trip [6, 7]. Under that model there is no reason to shorten a circuit’s depth below its Toffoli count. Two developments change the picture. High-rate quantum LDPC codes achieve fault-tolerant memory with an order of magnitude fewer physical qubits per logical qubit than the surface code [1, 2], and reconfigurable atom arrays offer significantly more physical qubits than other platforms, hence a roadmap to cheap additional logical qubits [3, 4]. Architectures for Shor’s algorithm on such hardware [4, 5] are limited by a slow logical cycle rather than by the qubit count, so it is worth paying meaningfully in space to save Toffoli depth. Cain et al. [4] make this move for the adders, replacing ripple-carry with carry-lookahead adders. Here we make it for the other ingredient of windowed arithmetic, the table lookup, and we keep the two changes separate so that the effect of each is visible.

Both factoring and the elliptic-curve discrete logarithm (ECDLP) are compiled today as long serial chains of windowed group operations (Figure 1). A control register is consumed a window of w bits at a time by the semiclassical Fourier transform, and each window addresses a table of

*Computations and drafting assisted by Claude (Anthropic). Code and data: `Titus/2026/EfficientDLWithQRAM`.

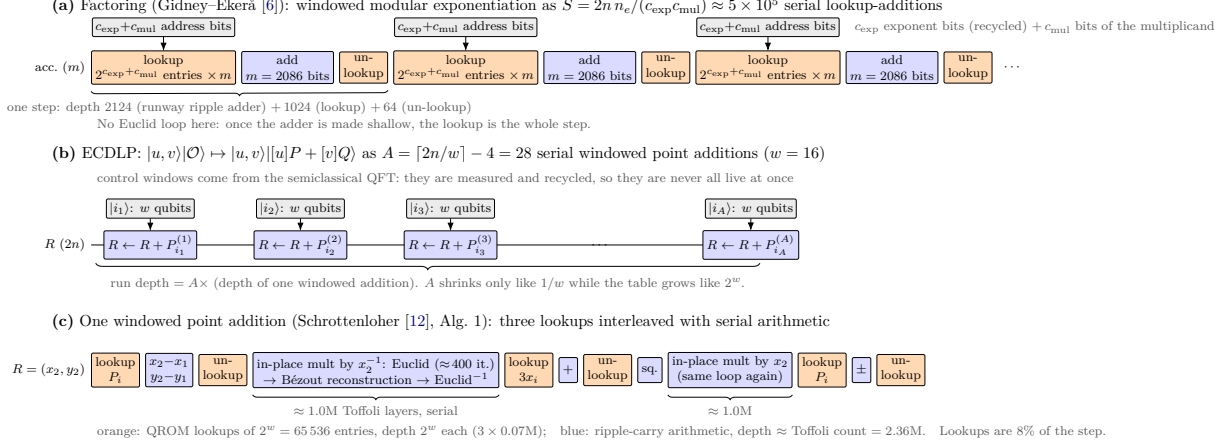


Figure 1: Shor’s algorithm as a serial chain of windowed operations. (a) Factoring in the Gidney–Ekerå compilation: lookup-additions on the accumulator. (b) ECDLP: windowed point additions, each driven by one w -qubit window of the semiclassical QFT. (c) One windowed point addition following Schrottenloher’s Algorithm 1: orange boxes are QROM lookups, blue boxes are ripple-carry arithmetic; the two in-place multiplications are $\approx 1.0\text{M}$ Toffoli layers each.

2^w precomputed constants that is read by a quantum lookup and then consumed by arithmetic. The standard lookup, QROM by unary iteration [11], costs 2^w Toffolis and 2^w sequential Toffoli layers. The SELECT-SWAP construction of Low et al. [8], with the clean-ancilla costs of Berry et al. [9] and generalised by Zhu et al. [10], replaces the single iteration by $2^w/\lambda$ group reads into λ registers followed by a logarithmic-depth swap network, dividing the lookup depth by λ for $\lambda - 1$ extra output-sized registers.

Our results are the following.

- **RSA-2048.** In the Gidney–Ekerå circuit [6] one step is a 2^{10} -entry lookup of 2086-bit words, a 2086-bit addition and an un-lookup, and the lookup is a third of the step’s depth. Changing only the lookup halves the run depth at about $10\times$ the qubits. Changing the adder as well, either by spacing the oblivious carry runways that Gidney and Ekerå already use more densely, or by substituting a carry-lookahead adder, brings the depth down by $20\times$ and $33\times$ respectively at $11\times$ the qubits (Section 3). The approximate-residue compilations [14, 15] are outside the scope of the lookup trade, but they admit a different one: Webster et al. [5] run Gidney’s circuit over ρ primes at once for 131 extra qubits per prime, and at equal qubit count that beats the lookup route by $2.5\times$ at 10k qubits and $5\times$ at 70k, with $1.5\text{--}2.5\times$ fewer Toffolis.
- **ECDLP-256.** We apply the lookup change, and nothing else, to the four published circuits whose windowed structure is specified: Schrottenloher [12], whose point-addition circuit we build from its implementation and find to have a Toffoli depth equal to $1.02\times$ its Toffoli count; the logical circuit of Häner et al. [27]; the low-depth circuit of Häner et al. [25]; and the depth-optimised circuit of Gu et al. [13]. In all four the lookups are 8–18% of the depth, a fraction we show to be $1/(w \ln 2)$ for an optimally chosen window, and the gain saturates at -30% to -40% for two orders of magnitude more qubits, because the modular inversion is a serial loop of hundreds of rounds that no lookup can shorten (Section 4). We also estimate what carry-lookahead adders would do for Schrottenloher’s circuit, the only one of the four built with ripple adders whose depth we can decompose: the arithmetic would shrink about $8\times$, the lookups would become 40% of a step, and the combination with shallow lookups would be the best ECDLP-256 front at every qubit budget, by $10\text{--}20\%$ over Gu et al. with $3\text{--}4\times$ fewer Toffolis.

2 Lookup constructions

A table lookup maps $|i\rangle|0\rangle \mapsto |i\rangle|x_i\rangle$ for an address $i \in [N]$, $N = 2^k$, and classical words x_i of b bits. All costs below are logical and assume all-to-all connectivity; the depth metric is Toffoli depth, the number of layers containing a Toffoli or AND gate, and Clifford fan-out is not charged. Table 1 collects the costs; Figure 2 sketches the three constructions. The first two are from the literature; the third is a small variant that we use for wide words.

construction	Toffolis	Toffoli depth	ancillas
QROM, unary iteration [11]	$N - 1$	$N - 1$	k
SELECT-SWAP $_{\lambda}$ [8, 9]	$N/\lambda - 1 + b(\lambda - 1)$	$N/\lambda - 1 + \log_2 \lambda$	$b(\lambda - 1) + \log_2(N/\lambda)$
λ parallel unary iterations (this work)	$N + \lambda$	$N/\lambda + 2 \log_2 \lambda$	$b(\lambda - 1)$
un-lookup, measurement based [9]	$N/k' + k'$	$N/k' + \log_2 k'$	k'

Table 1: Cost of a lookup of $N = 2^k$ entries of b bits. λ and k' are powers of two. $\lambda = 1$ is the QROM used by every resource estimate we compare against.

QROM by unary iteration. The address register is walked through all N values in order. Unary iteration [11] maintains a single “current address matches” flag qubit that is advanced from address j to $j + 1$ with one Toffoli, using a binary tree over the address bits so that only k ancillas are needed; the total is $N - 1$ Toffolis. When the flag for address j is set, the constant x_j is written into the b -bit output register by a fan-out of CNOTs targeting the positions where x_j has a 1. The Toffolis form a single chain, so the Toffoli depth equals the Toffoli count. Nothing about the lookup can be parallelised in this form: the depth is N whatever the word size.

SELECT-SWAP. Low et al. [8] split the address into $k - \log_2 \lambda$ high bits and $\log_2 \lambda$ low bits. A unary iteration over the high bits only, N/λ steps, writes at step g the λ consecutive words $x_{g\lambda}, \dots, x_{g\lambda+\lambda-1}$ into λ separate b -bit registers with one CNOT fan-out per register; this is the SELECT stage, and its depth is N/λ . The low address bits then control a swap network that brings register number i_{lo} to the top: $\log_2 \lambda$ layers, each swapping registers in parallel under the control of one address bit, $b(\lambda - 1)$ controlled swaps in total, each a Toffoli plus two CNOTs. The output register holds x_i and the other $\lambda - 1$ registers hold the remaining words of the group. Berry et al. [9] give the clean-ancilla accounting used in Table 1, and note that the $\lambda - 1$ side registers can be measured in the X basis immediately after the swap, so they are live only for the duration of the lookup; the measurement outcomes are kept for the un-lookup. Zhu et al. [10] embed this construction in a general architecture with a second parameter that trades T-count against error resilience on a planar layout; the depth–qubit exchange rate, $b(\lambda - 1)$ qubits per λ -fold reduction of the SELECT depth, is the same.

Parallel unary iterations. For the RSA case the words are $b = 2086$ bits wide and the table has $N = 1024$ entries, so the $b(\lambda - 1)$ swap Toffolis of SELECT-SWAP exceed the N Toffolis of the lookup itself as soon as $\lambda \geq 2$. We therefore use the following simple alternative. Compute from the $\log_2 \lambda$ high address bits a one-hot flag register of λ qubits ($\lambda - 1$ Toffolis, depth $\log_2 \lambda$). Then run λ unary iterations at the same time, one per flag, each over the N/λ addresses of its own chunk of the table and each writing into its own b -bit register; the flag is folded into the root of the iteration so that exactly one chunk actually writes. Finally XOR all λ registers into the output with $b(\lambda - 1)$ CNOTs in $\log_2 \lambda$ layers, and uncompute the flags. The Toffoli count is $\lambda \cdot N/\lambda + \lambda = N + \lambda$, the depth is N/λ for the iterations plus $2 \log_2 \lambda$ for the flags and fan-in, and the qubit cost is the same $b(\lambda - 1)$ side registers as SELECT-SWAP. It is worse than SELECT-SWAP in Toffolis when $b\lambda < N$ and better when $b\lambda > N$; the model takes the cheaper of the two for every (N, b, λ) . We are not aware of this variant being written down, although it is an obvious rearrangement of unary iteration.

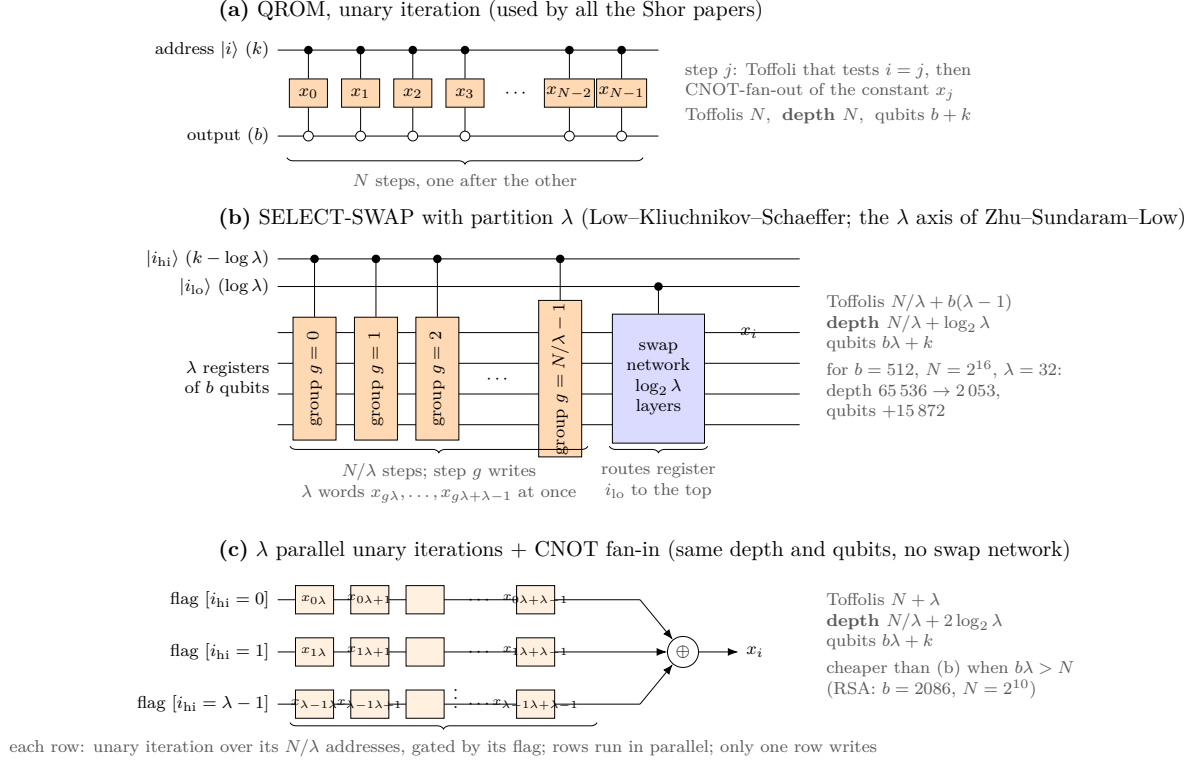


Figure 2: Three ways to execute a lookup of $N = 2^k$ entries of b bits. (a) QROM: serial unary iteration, depth N . (b) SELECT-SWAP with partition λ : N/λ group writes into λ registers, then a $\log_2 \lambda$ -layer swap network. (c) λ parallel unary iterations merged by a Clifford fan-in; same depth and qubits as (b) without the $b\lambda$ swap Toffolis.

Un-lookup. All three constructions are uncomputed with the measurement-based scheme of Berry et al. [9]: the output register is measured in the X basis, which leaves a phase depending on the address, and that phase is removed by a lookup of one bit per address, itself sped up by a partition k' : $N/k' + k'$ Toffolis with k' ancillas. Its depth $N/k' + \log_2 k'$ is included in every number below; for the ECDLP baseline it is 512 layers per un-lookup, and in the modified circuits we choose k' within the same qubit budget as λ .

Where the extra qubits live. The $\lambda - 1$ side registers exist only between the start of a lookup and its X -basis measurement, i.e. not during the arithmetic that follows. In the circuits below the lookups happen at points where the arithmetic’s own scratch space is idle, so the first few hundred side qubits are free (Sections 3 and 4 give the accounting per circuit); beyond that, each doubling of λ costs b more logical qubits for every lookup register that is live at once.

3 RSA-2048

Circuit. Gidney and Ekerå [6] factor an $n = 2048$ -bit modulus with $3n + O(n \lg n)$, in practice about 6.4k, logical qubits and 2.7×10^9 Toffolis in a single run (Ekerå–Håstad post-processing succeeds with probability $\geq 99\%$, so per-key and per-run counts coincide; Appendix B). The exponent, $n_e = 3029$ bits, is consumed by the semiclassical QFT and recycled. The exponentiation is a loop over exponent bits, each controlling a modular multiplication by a constant, and each multiplication is a loop over the bits of the quantum multiplicand, each controlling the addition of a constant; windowing reads c_{exp} exponent bits and c_{mul} multiplicand bits at a time, and since the constant to add depends on both, the two windows together address a single table of $2^{c_{\text{exp}} + c_{\text{mul}}}$ precomputed entries (Appendix A spells this out). The whole run is therefore

$S = 2nn_e/(c_{\text{exp}}c_{\text{mul}}) \approx 5.0 \times 10^5$ serial lookup-additions (Figure 1a), each of which looks up one of 2^{10} precomputed constants and adds it into the accumulator. The accumulator is held in the coset representation, which pads the n -bit value with $c_{\text{pad}} = 38$ extra bits so that a plain non-modular addition approximates a modular one; the looked-up word and the addition are therefore $m = n + c_{\text{pad}} = 2086$ bits wide. Per step, the Toffoli count is $2n + nc_{\text{pad}}/c_{\text{sep}} + 2^{10} \approx 5.2\text{k}$, of which the lookup is 19.5%, and the depth is $2c_{\text{sep}} + 2c_{\text{pad}} + 2^{10} = 3148$, of which the lookup is 33%. Here $c_{\text{sep}} = 1024$ is the spacing of the oblivious carry runways: the 2086-bit ripple-carry addition is cut into two pieces of 1024 bits that ripple in parallel, at a cost of c_{pad} runway qubits per piece. Gidney and Ekerå note that runways “can be introduced gradually” to shorten the adder, and that a faster adder pushes the optimal windows down by about a bit because the lookup then dominates.

What we change. We consider three families, in increasing order of intervention.

1. *Lookup only.* The adder is Gidney and Ekerå’s, with $c_{\text{sep}} = 1024$. Only the lookup partition λ and the window $k = c_{\text{exp}} + c_{\text{mul}}$ change.
2. *Denser carry runways.* Same adder family, with c_{sep} reduced from 1024 down to 16. The adder costs $2m + nc_{\text{pad}}/c_{\text{sep}}$ Toffolis and $2c_{\text{sep}} + 2c_{\text{pad}}$ depth, and needs $2(n/c_{\text{sep}})c_{\text{pad}}$ runway qubits. This is a parameter of the original construction, not a new adder.
3. *Carry-lookahead adder.* The ripple adder is replaced by the Draper et al. adder [24]: $10m$ Toffolis, depth $4\log_2 m$, $2m$ ancillas. This is a genuine substitution and it multiplies the Toffoli count by about five.

With $c_{\text{exp}} = c_{\text{mul}} = k/2$ the number of steps is $S(k) = 8nn_e/k^2$, and

$$D = S(k) (D_{\text{add}} + D_{\text{lookup}} + D_{\text{unlookup}}), \quad (1)$$

$$Q = 3m + \text{runways} + \max(\text{lookup side registers, adder ancillas, } k'). \quad (2)$$

The lookup side registers, adder ancillas and un-lookup ancillas are live at different times, so the extra qubit count is their maximum.

Results. Table 2 and Figure 3 give the Pareto fronts in depth for the three families, with the other RSA-2048 compilations as reference points. Changing only the lookup halves the depth: $\lambda = 32$ at $k = 12$ costs 71k qubits, $11\times$ the baseline, for $2.0\times$ less depth and $1.4\times$ more Toffolis, and the curve flattens near $2.8\times$ because the 2124-deep ripple adder remains. Denser runways remove that floor cheaply: runways every 64 bits with $\lambda = 2$ give $6\times$ for 35% more qubits, and runways every 16 bits with $\lambda = 32$ give $20\times$ at 71k qubits for $2\times$ the Toffolis. The carry-lookahead adder reaches $33\times$ at the same 71k qubits, and $50\times$ beyond 5×10^5 qubits, at $4\text{--}6\times$ the Toffolis. In the last two families the lookup is the majority of the step (Figure 5), so the shallow lookup is what carries the gain; without it a shallow adder alone is capped at about $3\times$ by the 2^{10} -deep QROM.

Residue-arithmetic compilations. Gidney [14] and Chevignard et al. [15] compute the exponentiation in a residue number system with approximate accumulation, reaching 1399 and 1730 logical qubits. Their exponent register (1280 resp. 1146 qubits) must be held rather than recycled, because every exponent bit serves as a lookup address once per residue prime (2.1×10^4 resp. 7.2×10^4 primes) [14, 15]. Their operations are 6-bit-address lookups of 32-bit words and 21–33-bit adders, i.e. 30–60 layers each, so shallow lookups have nothing to remove. They appear in Figure 3 as reference points with their per-key counts (9.2 shots and 40 runs respectively).

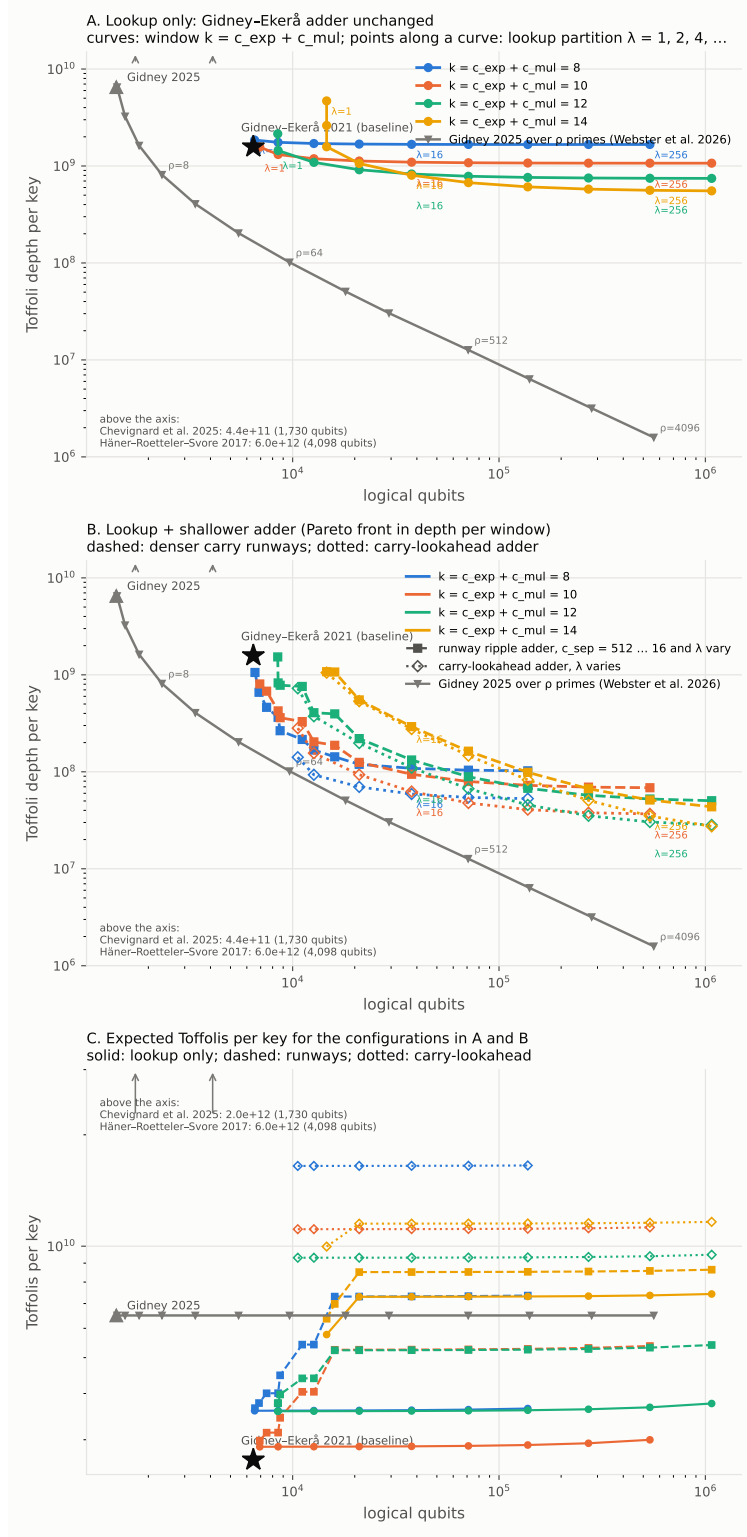


Figure 3: RSA-2048 with the Gidney-Ekerå chain. Curves are window sizes $k = c_{\text{exp}} + c_{\text{mul}}$; points along a curve are the lookup partition $\lambda = 1, 2, 4, \dots$. (A) Only the lookup is changed. (B) Lookup plus a shallower adder: dashed, denser carry runways (c_{sep} and λ vary along the Pareto front); dotted, carry-lookahead adder (λ varies). (C) Expected Toffolis per key of the same configurations. Grey triangles are the other RSA-2048 compilations, and the grey curve is Gidney’s residue circuit parallelised over ρ primes [5] (Appendix D).

family	qubits	k	c_{sep}	λ	depth / baseline	Toffolis / baseline
lookup only	8 506	10	1024	2	0.83	1.08
lookup only	21 024	12	1024	8	0.58	1.35
lookup only	71 088	12	1024	32	0.50	1.36
lookup only	538 354	14	1024	256	0.36	2.78
denser runways	8 698	8	64	2	0.17	1.69
denser runways	21 020	8	16	8	0.076	2.76
denser runways	71 086	10	16	32	0.050	1.99
denser runways	271 344	12	16	128	0.036	1.99
carry-lookahead	12 676	8	–	4	0.059	6.2
carry-lookahead	21 020	8	–	8	0.044	6.2
carry-lookahead	71 086	10	–	32	0.030	4.2
carry-lookahead	271 344	12	–	128	0.022	3.5
carry-lookahead	538 352	12	–	256	0.019	3.6

Table 2: RSA-2048, selected points of the Pareto fronts in Toffoli depth. Baseline [6] ($k = 10$, QROM, $c_{\text{sep}} = 1024$): 6452 qubits, depth 1.58×10^9 , 2.65×10^9 Toffolis per key.

Prime-level parallelism of the residue compilation. The residue circuit has its own depth-for-qubits trade, and it is the natural comparison for ours. Gidney’s circuit processes its $|P| \approx 2.1 \times 10^4$ primes one after another with a single working register of $N_w = 131$ qubits next to the $m = 1280$ -qubit exponent register. Webster et al. [5] observe that the primes are independent: with ρ working registers the outer loop runs ρ primes at once and the depth divides by ρ , while the exponent register is only read by lookups and can be shared by up to $\lceil m/w_1 \rceil = 214$ working registers, so that (their Eq. 22)

$$N(\rho) = \lceil \rho/214 \rceil m + \rho N_w, \quad D(\rho) = D_G/\rho, \quad (3)$$

at an unchanged Toffoli count of 6.5×10^9 per key, where $D_G \approx 6.5 \times 10^9$ is the depth of Gidney’s serial circuit. This curve is drawn in every panel of Figure 3, and Table 3 compares it with the best point of our fronts at the same qubit budget.

qubits	ρ	prime-parallel depth	best lookup-route depth	Toffolis (prime-parallel / lookup route)
9 664	64	1.0×10^8	2.7×10^8 (runways, $\lambda = 2$)	$6.5 \times 10^9 / 4.5 \times 10^9$
29 314	214	3.0×10^7	7.0×10^7 (carry-lookahead, $\lambda = 8$)	$6.5 \times 10^9 / 1.6 \times 10^{10}$
70 912	512	1.3×10^7	5.9×10^7 (carry-lookahead, $\lambda = 16$)	$6.5 \times 10^9 / 1.6 \times 10^{10}$
281 088	2048	3.2×10^6	3.5×10^7 (carry-lookahead, $\lambda = 128$)	$6.5 \times 10^9 / 9.4 \times 10^9$

Table 3: RSA-2048 at equal logical qubit budgets: Gidney’s residue circuit parallelised over ρ primes [5] against the best configuration of the lookup route (any family of Table 2).

The exchange rates explain the outcome. Another parallel prime costs $N_w = 131$ qubits and removes a full $1/\rho$ of the run, whereas another doubling of λ costs $m = 2086$ qubits per lookup register and removes a fraction of one step. Shallow lookups do not rescue the comparison inside the residue circuit either: its lookups are 6-bit and its adders 32-bit, so a step is already about 90 layers, and $\lambda = 8$ there would cost 224 qubits per working register for a $2\times$ step reduction, which is worse than spending the same qubits on more primes. The lookup route therefore remains the right tool for the recycled-exponent Gidney–Ekerå chain, where prime-level parallelism is not available, and below a few thousand qubits, where the residue circuit cannot yet afford a second working register; above that the prime-parallel residue circuit reaches lower depth with fewer Toffolis.

Other constructions. The $2n + 2$ -qubit circuit of Häner, Roetteler and Svore [19] (4098 qubits, $64n^3 \log_2 n \approx 6 \times 10^{12}$ Toffolis, serial) appears in Figure 3 as a reference point; Gidney’s $n + 2$ clean

plus $n - 1$ dirty qubit variant [20] has the same asymptotics without a concrete count. Regev’s algorithm and its space-efficient variants [21, 22, 23] use $O(n^{1.5})$ qubits and gates per run with about $\sqrt{n}$ runs; we know of no concrete logical estimate for $n = 2048$ to place on the plot. Cain et al. [4] run the Gidney–Ekerå chain with carry-lookahead adders and unary lookups, which is our carry-lookahead family at $\lambda = 1$; they report physical resources only, and note the lookup architecture of Zhu et al. [10] as a possible further step, which is the step taken here.

4 ECDLP-256

Constructions. We apply the lookup change to every published ECDLP-256 circuit whose windowed structure is specified, leaving the arithmetic of each untouched (Table 4). Schrottenloher [12] gives the point-addition circuit that reproduces Babbush et al. [7]: $A = \lceil 2n/w \rceil - 1 - \lfloor 48/w \rfloor = 28$ windowed point additions at $w = 16$, each with three QROM lookups of 2^w entries (two returning a point, $b = 2n$, one returning $3x$, $b = n$), two in-place modular multiplications implemented as a record-and-replay binary Euclidean loop, and a modular squaring (Figure 1c). We built this circuit with the author’s Qarton implementation (two-point window, so without lookups) and measured its Toffoli depth (Table 5); the depth exceeds the count by 2% only because Qarton weights classically-conditioned gates by their probability in the count but not in the depth. The arithmetic is a serial chain of ≈ 400 Euclidean iterations run forward, through the Bézout reconstruction and backward, each a ripple-carry operation of depth $\approx n$. The logical circuit of Häner et al. [27] has the same structure with a further-optimised arithmetic of 1.196×10^6 Toffolis per addition, again ripple-carry, so we take its count as its depth. The low-depth circuit of Häner et al. [25] uses Draper carry-lookahead adders, Kaliski inversion in $2n$ rounds, a signed 15-bit window and six lookups per window, since each of its three point lookups is uncomputed by a second lookup; its T-depth of $1.12 \cdot 2^{24}$ over 35 windows, less six lookups of 2^{14} T-layers each, gives the per-window arithmetic depth. Gu et al. [13] use their own carry-lookahead adders, Kaliski inversion, a signed 13-bit window and six lookups per window, and report a Toffoli depth of 1.09×10^7 over 40 windows.

construction	qubits	arithmetic Toffolis	arithmetic depth	lookups per window	lookup share
Schrottenloher 2026 [12]	1 195	2.30×10^6	2.36×10^6 (measured)	3×2^{16}	8%
Häner et al. 2026 [27]	1 446	1.20×10^6	1.20×10^6 (count)	3×2^{16}	14%
Häner et al. 2020 low-depth [25]	2 871	4.1×10^7 (T/4)	4.4×10^5 (T-depth)	6×2^{14}	18%
Gu et al. 2025 [13]	4 352	3.3×10^7	2.5×10^5	6×2^{12}	9%
Schrottenloher 2026 + carry-lookahead (estimate)	1 707	1.15×10^7	2.95×10^5 (estimate)	3×2^{16}	40%

Table 4: Per-window arithmetic of the four ECDLP-256 constructions (window register excluded) and the depth share of their lookups as published, plus the estimated carry-lookahead variant of Schrottenloher’s circuit.

variant	qubits	Toffolis	Toffoli depth
space-optimised, secp256k1	1195	2 304 135	2 359 541
gate-optimised, secp256k1	1443	1 799 437	1 839 947

Table 5: Arithmetic-only point addition circuit of [12], built with Qarton 1.0.0.

Why the lookups are a small fraction of the depth. Write the run depth as $D(w) = (2n/w)(D_A + c2^w)$ with D_A the arithmetic depth per addition and $c2^w$ the lookup depth. Adding a bit to the window removes one addition in about w , a $1/w$ saving, and doubles the lookup term. At the optimum $dD/dw = 0$, which gives $c2^w(w \ln 2 - 1) = D_A$, i.e.

$$\frac{\text{lookup depth}}{\text{step depth}} = \frac{c2^w}{D_A + c2^w} = \frac{1}{w \ln 2} \approx 9\% \text{ at } w = 16. \quad (4)$$

Any circuit whose window is tuned for depth (or, by the same argument, for Toffoli count) has this lookup share, independently of the arithmetic cost; the four constructions sit at 8–18%, the higher values where the uncompute is done by a second lookup. Consequently dividing the lookup depth by λ buys only $\log_2 \lambda$ extra window bits before the lookup is back to the same share, and each bit removes $1/w$ of the additions: the depth gain is about $\log_2(\lambda)/w$, logarithmic in λ and hence in qubits.

Model. For each construction, with T_A, D_A, Q_A its per-window arithmetic, k_{pt} point lookups ($b = 2n$), k_x coordinate lookups ($b = n$), $N = 2^w$ or 2^{w-1} entries for an unsigned or signed window, and L, L_d, L_q the lookup Toffolis, depth and side-register qubits of Section 2,

$$D_{\text{add}} = D_A + k_{\text{pt}} L_d(2n) + k_x L_d(n) + (k_{\text{pt}} + k_x) U, \quad D = A(w) D_{\text{add}}, \quad (5)$$

$$Q = \max(Q_A, \text{live data at the lookup} + L_q(2n)) + w, \quad (6)$$

and likewise for Toffolis; U is the measurement-based un-lookup. At a point lookup only the point registers and the lookup output are live, $4n$ qubits for the two Schrottenloher-type circuits and assumed $6n$ for the other two, so the first few hundred side qubits are free. For Schrottenloher and Häner 2026 the trailing-bits rule keeps the classically post-processed bits at 48 (Ekerå), reproducing $A = 28$ at $w = 16$; the other two use $\lceil 2n/w \rceil$ windows as published.

Results. Figure 4 shows, for each construction, the Pareto front in depth over the window and λ , starting from the circuit as published, with the remaining ECDLP-256 papers as reference points; Table 6 lists selected points. All four curves have the same shape. At the published window the lookups are 8–18% of the depth, so the first doubling of λ removes a few percent; the larger window that the shallow lookup then affords removes additions only logarithmically, and the fronts flatten at 0.60–0.69 of the published depth after two orders of magnitude more qubits. For Häner et al. 2020 the first –14% costs no qubits at all, since replacing the three uncompute lookups by measurement-based un-lookups and using $\lambda = 2$ fit inside the circuit’s idle registers. The Toffoli count moves with the depth (Figure 4B): a SELECT-SWAP lookup with a larger window costs fewer Toffolis than the published QROM while saving additions, so the counts drop by 10–30%, but spacetime volume rises in every configuration. Per-window detail for Schrottenloher’s circuit is in Appendix C.

Carry-lookahead adders in Schrottenloher’s circuit (estimate). The two constructions built for depth owe their position in Figure 4 to their adders rather than to their inversion algorithm, which is the heavier Kaliski loop. Schrottenloher’s circuit combines the lighter record-and-replay Euclidean loop with ripple-carry adders, and its measured depth is a chain of about 9 000 operations of depth $\approx n$. Replacing each by a Draper et al. carry-lookahead operation [24] of depth $\approx 4 \log_2 n$ would divide the arithmetic depth by about 8, multiply its Toffolis by about 5 ($10n$ against $2n$ per adder) and add $2n$ ancillas: an estimated 2.95×10^5 layers per addition at 1 707 qubits (Table 4, last row). We have not built this circuit; some of the operations are 50-bit comparisons or small constant additions that are already shallow, so the true factor is somewhat below 8. Two things follow. As published, with QROM lookups at $w = 16$, the variant would run in 1.25×10^7 layers at 1 723 qubits, already below Gu et al.’s 1.06×10^7 at 4 365 once qubits are compared. And the lookups become 40% of a step, so the shallow lookup finally pays: the dashed curve in Figure 4 and the last block of Table 6 give 8.9×10^6 at 4.6k qubits and 7.9×10^6 at 33k, 10–20% below Gu et al. at the same budgets with 3–4 $\times$ fewer Toffolis. The floor does not move, however: the Euclidean loop is still ≈ 400 data-dependent iterations run three times per multiplication, so a point addition cannot go below roughly 2 400 sequential adder-equivalents, of order 10^5 layers, without a different inversion algorithm. Qarton parameterises the arithmetic by adder class, so a measured version of this estimate is a modest extension of the run behind Table 5.

construction	qubits	w	λ	depth / published	Toffolis / published
Schrottenloher 2026 (1 211 qubits, depth 7.1×10^7)	8 748	20	16	0.82	0.83
	33 326	22	64	0.75	0.78
	131 632	24	256	0.69	0.79
Häner et al. 2026 (1 462 qubits, depth 3.8×10^7)	8 748	20	16	0.82	0.84
	33 322	20	64	0.75	0.81
	131 628	22	256	0.69	0.91
Häner et al. 2020 low-depth (2 886 qubits, depth 1.9×10^7)	2 886	15	2	0.86	1.00
	9 247	18	16	0.72	0.83
	33 825	20	64	0.65	0.74
	132 131	22	256	0.60	0.69
Gu et al. 2025 (4 365 qubits, depth 1.1×10^7)	9 243	16	16	0.78	0.80
	33 825	20	64	0.73	0.65
	132 127	20	256	0.65	0.67
Schrottenloher + carry-lookahead, estimate (1 723 qubits, depth 1.25×10^7)	4 645	16	8	0.71	0.99
	33 322	20	64	0.63	0.82
	131 624	20	256	0.57	0.84

Table 6: ECDLP-256, selected points of the Pareto fronts in depth, lookups changed only. The published circuit (QROM, own window and un-lookup) is the reference in each block; for the last block the reference is the estimated variant with QROM lookups.

5 Discussion

Figure 5 summarises the mechanism. Shallow lookups exchange $b(\lambda - 1)$ qubits for a λ -fold lookup speed-up, and the value of that exchange is set by the lookup’s share of one serial step. In the Gidney–Ekerå factoring chain that share is a third to begin with and becomes the whole step once the adder is shortened, so the lookup change alone halves the depth and, combined with denser runways or a carry-lookahead adder, delivers one to one-and-a-half orders of magnitude for a few times the qubits. That is nevertheless not the best use of extra qubits for RSA-2048: the residue compilation run over many primes at once [5] buys a full unit of parallelism for 131 qubits, against 2086 qubits per doubling of λ , and it wins at every budget above a few thousand qubits with fewer Toffolis. The lookup route is the one to use when the exponent must be recycled rather than held. In ECDLP the share is $1/(w \ln 2)$ by construction and the inversion loop is inherently sequential, so the trade saturates at 30–40% of the depth in all four published constructions, including the two built for depth. The adder is the larger lever there, and our estimate for Schrottenloher’s circuit with carry-lookahead adders suggests that combining the two would set the ECDLP-256 front; measuring that circuit is the natural next step. Two caveats apply to both. The depth reductions assume all-to-all connectivity and count Toffoli layers only; on a nearest-neighbour layout the lookup carries extra logarithmic factors [10]. And a shallower circuit with the same or larger Toffoli count needs proportionally higher magic-state throughput, so among the RSA options the runway family, which roughly doubles the Toffoli count, is the natural first step, and the carry-lookahead family is for machines where CCZ states are cheap relative to logical cycles.

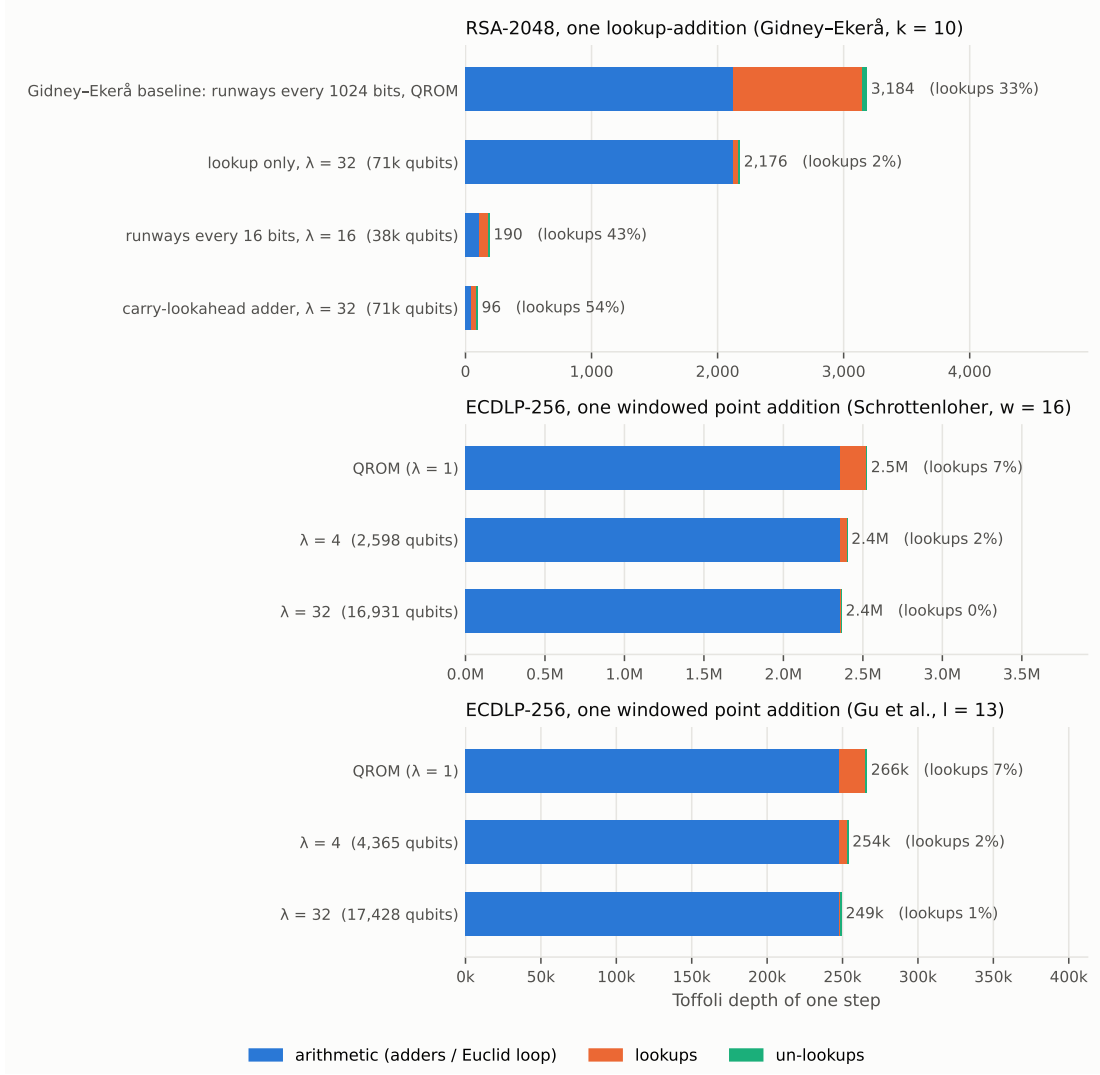


Figure 5: Toffoli depth of one serial step, split into arithmetic, lookups and un-lookups. Top: one Gidney–Ekerå lookup-addition, baseline and the three families of Section 3. Middle: Schrottenloher’s windowed point addition. Bottom: Gu et al.’s (the two Häner constructions behave like these two).

A Windowed modular exponentiation and the two window sizes

This appendix spells out the structure behind c_{exp} , c_{mul} and the step count S of Section 3, following Gidney and Ekerå [6].

The computation. The algorithm needs $|e\rangle|g^e \bmod N\rangle$ for a superposed exponent e of n_e bits, a fixed base g and the n -bit modulus N . With $e = \sum_j e_j 2^j$,

$$g^e \bmod N = \prod_j (g^{2^j})^{e_j} \bmod N, \quad (7)$$

and every factor $g^{2^j} \bmod N$ is a classical constant. Unwindowed, the accumulator starts at 1 and, for each exponent bit e_j , is multiplied by the constant g^{2^j} under the control of e_j : n_e controlled modular multiplications by classical constants.

One multiplication by a constant. For a quantum register x and a classical constant C , schoolbook multiplication gives $x \cdot C = \sum_i x_i (C2^i)$, so a multiplication is a loop over the bits of the multiplicand, each step adding the classical constant $C2^i \bmod N$ into an output register under the control of the quantum bit x_i . Since the product must replace the accumulator, it is done out of place twice: $y \leftarrow x \cdot C$ into a fresh register, then $x \leftarrow x - y \cdot C^{-1}$ to clear the old value, then a swap. One exponent bit therefore costs about $2n$ controlled additions, and the unwinded run about $2n n_e$ of them.

The exponent window c_{exp} . Read c_{exp} consecutive exponent bits $e_j, \dots, e_{j+c_{\text{exp}}-1}$ together; their value $v \in [2^{c_{\text{exp}}}]$ selects

$$\prod_{t < c_{\text{exp}}} (g^{2^{j+t}})^{e_{j+t}} = g^{v 2^j} \bmod N, \quad (8)$$

one of $2^{c_{\text{exp}}}$ precomputable constants. The number of multiplications falls from n_e to n_e/c_{exp} . These c_{exp} qubits are the window of the semiclassical Fourier transform: prepared, used as controls for one windowed multiplication, rotated, measured and recycled, so the exponent never needs n_e live qubits.

The multiplication window c_{mul} . Inside a multiplication by the selected constant C , read c_{mul} consecutive multiplicand bits $x_i, \dots, x_{i+c_{\text{mul}}-1}$ together; their value $u \in [2^{c_{\text{mul}}}]$ selects

$$\sum_{t < c_{\text{mul}}} x_{i+t} C 2^{i+t} = u C 2^i \bmod N, \quad (9)$$

one of $2^{c_{\text{mul}}}$ precomputable constants. The number of additions per multiplication falls from $2n$ to $2n/c_{\text{mul}}$. Unlike the exponent window these bits are data, part of the accumulator, and are only read as an address.

One table for both windows. The constant to add at position (i, j) depends on both u and v : it is $u g^{v 2^j} 2^i \bmod N$. Rather than first selecting a multiplier from v and then multiplying, the whole product is precomputed for every pair (u, v) . The table for one (i, j) has $2^{c_{\text{exp}}+c_{\text{mul}}}$ entries, its address register is the concatenation of the c_{exp} exponent qubits and the c_{mul} multiplicand qubits, and each entry is an n -bit constant padded to $m = n + c_{\text{pad}}$ bits for the coset representation, in which a plain addition approximates a modular one. One lookup-addition is thus: look up the word at address (v, u) , add it into the accumulator, un-look it up. Each step replaces $c_{\text{exp}} c_{\text{mul}}$ of the original controlled additions, so

$$S = \frac{n_e}{c_{\text{exp}}} \cdot \frac{2n}{c_{\text{mul}}} = \frac{2n n_e}{c_{\text{exp}} c_{\text{mul}}} \approx 5.0 \times 10^5 \quad \text{at } c_{\text{exp}} = c_{\text{mul}} = 5, \quad (10)$$

the factor 2 being the compute-and-uncompute pair of multiply-adds.

Why $c_{\text{exp}} = c_{\text{mul}} = 5$. Per step the addition costs about $2n$ Toffolis regardless of the windows, while the lookup costs $2^{c_{\text{exp}}+c_{\text{mul}}}$. Enlarging a window by one bit removes a fraction $1/c$ of the steps (about 20% at $c = 5$) and doubles the lookup, so the Toffoli-optimal balance puts the lookup at roughly a fifth of the step, which is $c_{\text{exp}} + c_{\text{mul}} = 10$; in depth the baseline step is 2124 layers for the runway-split adder plus 1024 for the lookup. The partition λ of Section 2 shortens the lookup part of each step without changing S ; enlarging $k = c_{\text{exp}} + c_{\text{mul}}$ shrinks S quadratically in the window bits at an exponential cost in the table, which becomes affordable only once λ has made the lookup cheap; the adder options shorten the other part of the step.

B Per-run and per-key counts

All Toffoli counts and depths in the figures are expected values per key. Table 7 lists the per-run numbers and the expected number of runs used for the conversion. For Gidney and Ekerå [6] the algorithm itself needs one run; their Table 3 also quotes a 31% retry risk at surface-code distance 27, which is a physical-layer budget choice (a distance of 29 brings it near zero) and is not included here.

circuit	metric	per run	runs per key	source
Gidney–Ekerå 2021 [6]	Toffolis	2.7×10^9	1	Ekerå–Håstad, $\geq 99\%$ post-processing
Gidney 2025 [14]	Toffolis	7.1×10^8	9.2	$s + 1 = 9$ shots plus bad shots (Table 5)
Chevignard et al. 2025 [15]	Toffolis / depth	$2^{35.54} / 2^{33.36}$	40	stated average
Schrottenloher 2026 [12]	Toffolis	$2^{26.11}$	1	“needs to run only once”
Babbush et al. 2026 [7]	Toffolis	$2^{26.27} / 2^{25.94}$	1	same algorithm
Gu et al. 2025 [13]	Toffolis / depth	$1.33 \times 10^9 / 1.09 \times 10^7$	1	“a single instance”
Häner et al. 2026 [27]	Toffolis	3.9×10^7	1	63% is the physical-level success
Chevignard et al. 2026 [16]	Toffolis	$2^{38.98}$	22	stated average
Litinski 2023 [26]	Toffolis	1.09×10^8 per key	–	reported per key

Table 7: Conversion from per-run to per-key counts.

C Per-window detail

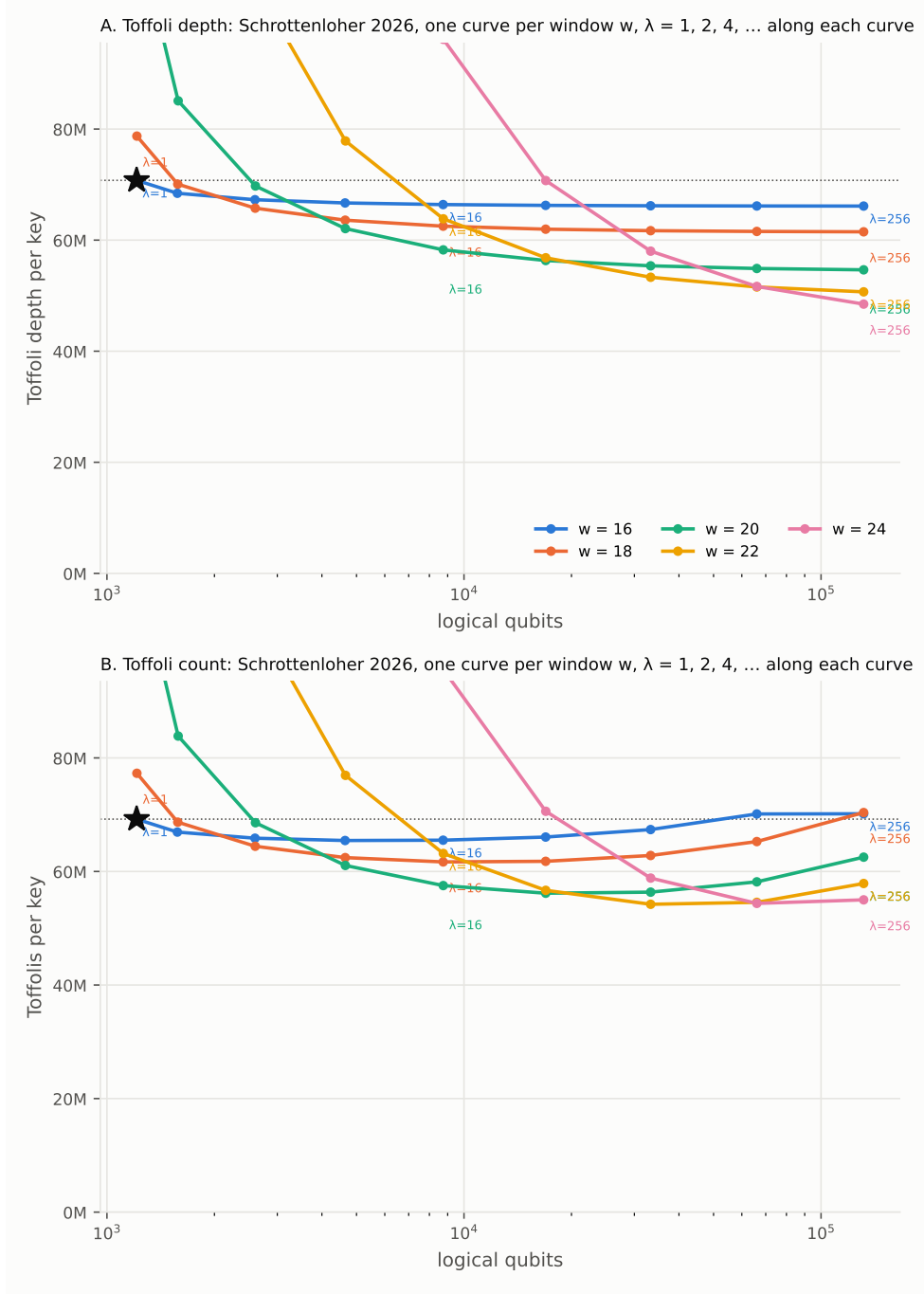


Figure 6: Schrottenloher’s circuit, one curve per window size w with $\lambda = 1, 2, 4, \dots$ along each curve, (A) depth and (B) Toffolis per key. The envelope of these curves is the Schrottenloher front of Figure 4.

D Reference points in the figures

Where a paper reports a Toffoli depth (or T-depth) we plot it; where it reports only a Toffoli count for a ripple-carry circuit we plot the count as the depth, since such circuits are serial (Table 5 shows the two agree within 2% for Schrottenloher’s circuit). T-counts are divided by four to compare with Toffoli counts.

circuit	qubits	depth / key	Toffolis / key	basis
Roetteler et al. 2017 [18]	2 330	1.16×10^{11}	1.26×10^{11}	Toffoli depth reported
Häner et al. 2020 low-width [25]	2 124	2.1×10^9	1.8×10^9	T-depth; T-count/4
Häner et al. 2020 low-depth [25]	2 871	1.9×10^7	1.4×10^9	T-depth; T-count/4
Litinski 2023 [26]	6 000	1.1×10^8	1.1×10^8	count as proxy
Babbush et al. 2026 [7], space opt.	1 191	8.1×10^7	8.1×10^7	count as proxy
Babbush et al. 2026 [7], gate opt.	1 441	6.5×10^7	6.5×10^7	count as proxy
Chevignard et al. 2026 [16]	1 193	1.2×10^{13}	1.2×10^{13}	22 runs; count as proxy
Luo et al. 2026 [17]	835	2.2×10^9	2.2×10^9	count as proxy
Häner et al. 2026 [27]	1 462	3.9×10^7	3.9×10^7	count as proxy; 7.5×10^7 layers measured
Gu et al. 2025 [13]	4 352	1.09×10^7	1.33×10^9	Toffoli depth reported
Gidney 2025 [14]	1 399	6.5×10^9	6.5×10^9	serial ops; count as proxy
Chevignard et al. 2025 [15]	1 730	4.4×10^{11}	2.0×10^{12}	depth reported
Häner, Roetteler, Svore 2017 [19]	4 098	6.0×10^{12}	6.0×10^{12}	serial; count as proxy
Gidney 2025 over ρ primes [5]	$\lceil \rho/214 \rceil 1280 + 131\rho$	$6.5 \times 10^9/\rho$	6.5×10^9	Eq. 22 of [5]; depth as for Gidney 2025

Table 8: Literature points shown in Figures 3 and 4. Häner et al. 2020 low-depth, Häner et al. 2026 and Gu et al. 2025 appear in Figure 4 as the starting points of modified curves rather than as grey points.

References

- [1] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, T. J. Yoder, *High-threshold and low-overhead fault-tolerant quantum memory*, Nature 627, 778 (2024).
- [2] Q. Xu, J. P. Bonilla Ataides, C. A. Pattison, N. Raveendran, D. Bluvstein, J. Wurtz, B. Vasić, M. D. Lukin, L. Jiang, H. Zhou, *Constant-overhead fault-tolerant quantum computation with reconfigurable atom arrays*, Nature Physics 20, 1084 (2024).
- [3] D. Bluvstein et al., *Logical quantum processor based on reconfigurable atom arrays*, Nature 626, 58 (2024).
- [4] M. Cain, Q. Xu, R. King, L. R. B. Picard, H. Levine, M. Endres, J. Preskill, H.-Y. Huang, D. Bluvstein, *Shor’s algorithm is possible with as few as 10,000 reconfigurable atomic qubits*, arXiv:2603.28627 (2026).
- [5] P. Webster, L. Berent, O. Chandra, E. T. Hockings, N. Baspin, F. Thomsen, S. C. Smith, L. Z. Cohen, *The Pinnacle architecture: reducing the cost of breaking RSA-2048 to 100 000 physical qubits using quantum LDPC codes*, arXiv:2602.11457 (2026).
- [6] C. Gidney, M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum 5, 433 (2021).
- [7] R. Babbush et al., *Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations*, arXiv:2603.28846 (2026).
- [8] G. H. Low, V. Kliuchnikov, L. Schaeffer, *Trading T gates for dirty qubits in state preparation and unitary synthesis*, Quantum 8, 1375 (2024).
- [9] D. W. Berry, C. Gidney, M. Motta, J. R. McClean, R. Babbush, *Qubitization of arbitrary basis quantum chemistry leveraging sparsity and low rank factorization*, Quantum 3, 208 (2019).
- [10] S. Zhu, A. Sundaram, G. H. Low, *Unified architecture for quantum lookup tables*, Phys. Rev. Research 7, 043230 (2025), arXiv:2406.18030.
- [11] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, H. Neven, *Encoding electronic spectra in quantum circuits with linear T complexity*, Phys. Rev. X 8, 041015 (2018).
- [12] A. Schrottenloher, *Optimized point addition circuits for elliptic curve discrete logarithms*, arXiv:2606.02235 (2026).
- [13] Q. Gu, H. Ye, J. Chen, X. Ma, *Resource analysis of Shor’s elliptic curve algorithm with an improved quantum adder on a two-dimensional lattice*, arXiv:2510.23212 (2025).
- [14] C. Gidney, *How to factor 2048 bit RSA integers with less than a million noisy qubits*, arXiv:2505.15917 (2025).

- [15] C. Chevignard, P.-A. Fouque, A. Schrottenloher, *Reducing the number of qubits in quantum factoring*, CRYPTO 2025, LNCS 16001, 384–415; ePrint 2024/222.
- [16] C. Chevignard, P.-A. Fouque, A. Schrottenloher, *Reducing the number of qubits in quantum discrete logarithms on elliptic curves*, EUROCRYPT 2026, ePrint 2026/280.
- [17] H. Luo, Z. Yang, J. Luo, Z. Wang, Y. Su, X. Sun, L. Li, T. Li, *Quantum algorithm for elliptic curve discrete logarithms with space-efficient point addition*, arXiv:2607.13816 (2026).
- [18] M. Roetteler, M. Naehrig, K. M. Svore, K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, ASIACRYPT 2017, LNCS 10625, 241–270, arXiv:1706.06752.
- [19] T. Häner, M. Roetteler, K. M. Svore, *Factoring using $2n + 2$ qubits with Toffoli based modular multiplication*, Quantum Inf. Comput. 17, 673–684 (2017), arXiv:1611.07995.
- [20] C. Gidney, *Factoring with $n + 2$ clean qubits and $n - 1$ dirty qubits*, arXiv:1706.07884 (2017).
- [21] O. Regev, *An efficient quantum factoring algorithm*, arXiv:2308.06572 (2023).
- [22] S. Ragavan, V. Vaikuntanathan, *Space-efficient and noise-robust quantum factoring*, arXiv:2310.00899 (2024).
- [23] M. Ekerå, J. Gärtner, *Extending Regev’s factoring algorithm to compute discrete logarithms*, PQCrypto 2024, LNCS 14772, 211–242 (2024), arXiv:2311.05545.
- [24] T. G. Draper, S. A. Kutin, E. M. Rains, K. M. Svore, *A logarithmic-depth quantum carry-lookahead adder*, Quantum Inf. Comput. 6, 351–369 (2006), quant-ph/0406142.
- [25] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, M. Soeken, *Improved quantum circuits for elliptic curve discrete logarithms*, PQCrypto 2020, LNCS 12100, 425–444, arXiv:2001.09580.
- [26] D. Litinski, *How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates*, arXiv:2306.08585 (2023).
- [27] T. Häner et al., *Computing 256-bit elliptic curve discrete logarithms in 26 days on a fault-tolerant trapped-ion quantum computer with 20,000 qubits*, arXiv:2609.05625 (2026).